Obstacle Avoidance in Autonomous Navigation with ROS 2

Md Mushfiqul Islam
islam.md-mushfiqul@stud.fra-uas.de
Matriculation Number: 1438422
Frankfurt University of Applied Sciences
Information Technology (M.Eng.)

Rezoyana Islam Bonna
rezoyana.bonna@stud.fra-uas.de
Matriculation Number: 1502329
Frankfurt University of Applied Sciences
Information Technology (M.Eng.)

***Abstract* — Autonomous robots that work in changing environments need strong local planners that can avoid obstacles in real time. This project tests and uses two ways to plan for a TurtleBot3 Burger robot in ROS 2 Jazzy with Gazebo Harmonic simulation: the Model Predictive Path Integral (MPPI) controller as the main trajectory optimizer and the Regulated Pure Pursuit Controller (RPP) as the DWA-equivalent baseline. The simulations were designed with four different scenarios: one with static obstacles, another with narrow passages, a third with dynamic obstacles, and finally, a mixed environment. Data will be collected systematically over eight experiment runs.**

The results show that the MPPI method is better at achieving its goals and adjusting its actions compared to the RPP method. This shows that predictive trajectory optimization is better than reactive velocity-sampling methods in messy and limited spaces. The project gives you a repeatable ROS 2 simulation framework with automatic metrics collection, which lets you compare local planners in a structured way across different obstacle scenarios.

Keywords — ***obstacle avoidance, ROS 2, MPPI controller, local planner, Nav2, Gazebo Harmonic, TurtleBot3, autonomous navigation***

I. Introduction

Autonomous mobile robots need to be able to safely move through places with both moving and stationary obstacles. The local planner is the part of the robot that makes commands for speed that keep the robot on a path that has been planned out ahead of time while avoiding obstacles that are close by in real time. The choice of local planner has a direct effect on how well navigation works, how efficient paths are, and how safe they are. The Nav2 navigation stack in Robot Operating System 2 (ROS 2) lets you use different local planner plugins that can be switched out. The Dynamic Window Approach (DWA) is a traditional method that finds possible velocity commands by sampling the robot's velocity space and scoring trajectories using a cost function. In contrast, the Timed Elastic Band (TEB) is a more advanced planning method. It represents the trajectory as an elastic band of poses, which are optimized over time. This makes paths smoother and makes it easier to deal with kinematic constraints. This project, which is part of the Autonomous and Intelligent Systems (AIS) module at Frankfurt University of Applied Sciences, tests and compares two local planning methods in a simulated ROS 2 environment. Model Predictive Path Integral (MPPI) is the main algorithm used for planning tasks.

This method, which uses a stochastic approach to improve trajectories, evaluates many possible paths and selects the best one. This selection is based on a cost function that includes weighted factors. While MPPI's trajectory optimization shares similarities with the Time Elastic Band (TEB) approach, it distinguishes itself by employing stochastic sampling rather than deterministic graph optimization. It's also a part of ROS 2 Jazzy. The Regulated Pure Pursuit Controller (RPP) serves as the foundation for the DWA-equivalent. Four simulation scenarios are assessed: static obstacles, narrow passages, dynamic obstacles, and a hybrid environment integrating both. We systematically record quantitative metrics like path length, time to goal, near-collision count, minimum obstacle distance, trajectory smoothness, and replanning frequency across all eight experiment runs. This gives us a structured and repeatable way to compare the results.

II. Literature Review

Mobile robots need to be able to navigate on their own in order to move safely in new or changing environments without support from people. One of the most important parts of autonomous navigation is avoiding obstacles. This is because it helps robots recognize obstacles and modify their direction in real time. A lot of people use the Robot Operating System (ROS) to develop robots since it is modular and offers a lot of capabilities for sensing, planning, and control. With the release of ROS 2, autonomous navigation has gotten a lot better because to new capabilities including real-time communication, increased dependability, and support for distributed systems. Because of these modifications, ROS 2 is excellent for complex robotic applications like service robotics and industrial automation [1].

There are usually modules for global planning, local planning, and motion control in the navigation stack of ROS-based navigation systems. The global planner uses a map of the environment to find a path for the robot, moving it from its current position to its goal. In contrast, the local planner creates possible paths in real time, taking into account the robot's movement limits and nearby obstacles. Costmaps are frequently employed to highlight obstacles, as identified by sensors such as cameras and LiDAR, that impede movement. These costmaps assist the planner identify safe, collision-free paths when traversing [1] by giving different areas of the environment risk values. The Dynamic Window Approach (DWA) is a commonly used local planner in navigation systems that use the Robot Operating System (ROS). This method works by creating a dynamic window. This window, which is based on the robot's current speed and acceleration limits, helps to explore possible velocity commands. Each potential path is then evaluated based on how close it is to the goal, how clear it is of obstacles, and how quickly it moves forward. Although DWA is fast and works in real time, it has a key limitation: it only considers obstacles as stationary when they are detected. As a result, it doesn't

predict where moving obstacles will be in the future, making it less effective in situations with dynamic obstacles [1].

The Timed Elastic Band (TEB) planner is a highly regarded local planning algorithm within ROS navigation frameworks, primarily because it uses optimization to create trajectories. The TEB algorithm models a robot's movement as a series of poses interconnected by elastic bands. These bands are subject to continuous optimization, taking into account the robot's kinematic characteristics, the presence of obstacles, and temporal limitations. As a result, the robot can dynamically modify its trajectory while ensuring a safe separation from potential collisions. TEB demonstrates superior performance compared to conventional planners in managing non-holonomic constraints and generating smoother trajectories, thereby making it an appropriate selection for mobile robots operating in dynamic environments [2].

The Timed Elastic Band (TEB) algorithm draws its conceptual foundation from earlier mobile robot path planning techniques that utilized elastic bands. This method leverages temporal data to represent the desired trajectory, thereby enabling the planner to perform optimization while accounting for speed and acceleration limitations. The trajectory optimization problem is framed as a multi-objective optimization task, aiming to discover a solution that concurrently fulfills all imposed constraints, such as evading obstacles, minimizing travel duration, and ensuring smooth motion. Data demonstrated the method's capacity to rapidly generate collision-free paths and function in real-time, thereby establishing its suitability for robotic navigation applications [3].

Recent research has investigated improvements to the Timed Elastic Band (TEB) approach, with the goal of increasing robustness in complex environments. A significant development in this area is the Resilient Timed Elastic Band (RTEB) planner. This planner incorporates TEB and other methods, including Hybrid A*, to produce alternative trajectories when the primary path becomes impractical. The planner's ability to navigate through both crowded and unfamiliar environments is improved by using soft constraints and a dynamic Voronoi-based method for modeling obstacles. Tests show that these changes can reduce both the time it takes to get somewhere and the distance traveled, while still ensuring safe obstacle avoidance [4].

Model Predictive Control (MPC) constitutes another significant area of local trajectory planning. These techniques represent an advancement over traditional navigation systems by anticipating the future positions of moving obstacles. Rather than reacting to sensor data, these technologies utilize probabilistic models to forecast the future displacement of obstructions, subsequently modifying the robot's trajectory accordingly. Simulation studies have shown that these methodologies improve safety and enable navigation within environments characterized by dynamic barriers [5].

The Model Predictive Path Integral (MPPI) controller represents a notable advancement in the domain of sampling-based local trajectory planning. This stochastic optimal control methodology generates a multitude of potential trajectories by incorporating Gaussian noise into the current control sequence. Subsequently, each trajectory is evaluated against a composite cost function, and the optimal control update is derived through an information-theoretic weighted average. This method proves particularly effective in dynamic and uncertain environments, owing to its ability to handle non-linear dynamics and cost functions that are not suitable for gradient-based methods.

Williams et al. developed the theoretical foundation for MPPI and demonstrated its robust real-time performance in autonomous driving, showing that the approach can operate at speeds surpassing the vehicle's friction limits [6]. MPPI is a local planner plugin for ROS 2 Jazzy that comes with the Nav2 stack. It is a modern and computationally efficient alternative to planners that use optimization. Research has shown that autonomous robots need to have good local trajectory planning algorithms so they can avoid obstacles. People still like reactive methods like DWA because they are easy to use. Conversely, optimization-driven planners such as TEB and sampling-based predictive controllers like MPPI offer advantages due to their capacity to generate smoother trajectories, their adaptability to dynamic obstacles, and their compatibility with the modular architecture of ROS 2 navigation systems. This investigation builds upon these observations by employing and comparing MPPI with the DWA-equivalent Regulated Pure Pursuit Controller across four disparate obstacle scenarios within a ROS 2 Jazzy simulation environment.

.

III. Methodology

A. System Architecture

The simulation system uses ROS 2 Jazzy, Gazebo Harmonic as the physics engine, and RViz2 as the visualization tool. The TurtleBot3 Burger differential-drive robot is the simulated platform. It has a 360-degree LiDAR sensor (LDS-01) that sends laser scan data to the /scan topic at a rate of 5 Hz. The Nav2 navigation stack sets up the planning and control framework. It has the NavFn global planner and two local planners that can be switched out for comparison: the MPPI controller as the main planner and the Regulated Pure Pursuit Controller (RPP) as the DWA-equivalent baseline. A static map-to-odom coordinate transform gives the simulation ground-truth localization. This gets rid of pose estimate uncertainty and makes sure that variances in performance are due to the local planners and not localization mistakes.

To make Gazebo Harmonic work with other programs, a compatibility fix was needed. The Nav2 stack sends velocity commands as geometry_msgs/Twist, but Gazebo Harmonic wants them as geometry_msgs/TwistStamped. A special bridge node in the launch file fixes this problem without the user having to do anything at runtime.

B. Simulation Scenarios

Four different Gazebo world files were made to test each planner in more and more difficult situations. All of the scenarios start the robot at (0, 0) and end it at (4.0, 0.0), which makes it easy to compare metrics across all of them. In the static scenario, there are five fixed box obstacles that are 0.5 x 0.5 x 0.5 m in size between the starting point and the goal. This tests the planner's ability to find a way around known stationary items without hitting them.

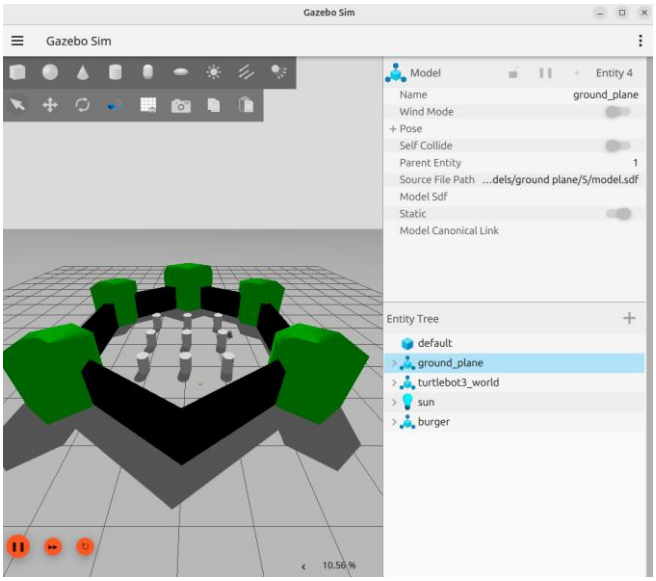

Fig. 1. Gazebo Harmonic simulation environment with TurtleBot3 Burger in the static obstacle scenario.

In this picture, you can see the Gazebo simulation environment for this scenario. The TurtleBot3 Burger robot is in the right corner of the arena, and the green obstacle geometry of the TurtleBot3 default world and the white cylindrical obstacles are around it.

In the restricted passage scenario, two pairs of walls are built to make sequential corridors with a 0.6 m gap. This is hard because the robot is 0.44 m in diameter. This situation challenges the planner's ability to go around in confined spaces where even slight mistakes in positioning can cause a crash.

In the dynamic scenario, two-cylinder obstacles move along predetermined waypoint courses at rates of 0.4 m/s and 0.3 m/s, respectively. A separate ROS 2 node controls these speeds. This situation tests how well the robot can design its path again in real time when moving items cross its planned path.

The mixed scenario has two static box obstacles and two dynamic cylinders at the same time. This is the hardest test of how well someone can avoid obstacles.

C. *Planner Configuration*

The MPPI controller was set up with a batch size of 2,000 trajectory samples, a time horizon of 56 steps at dt = 0.05 s, and eight cost critics: ConstraintCritic, ObstaclesCritic, GoalCritic, GoalAngleCritic, PathAlignCritic, PathFollowCritic, PathAngleCritic, and PreferForwardCritic. The weight of the obstacle critic was set to 20.0, and the distance for the collision margin was set to 0.1 m. The TurtleBot3 Burger hardware constraints set the maximum linear velocity to 0.22 m/s and the maximum angular velocity to 1.0 rad/s. The RPP baseline had a desired linear speed of 0.22 m/s and a lookahead distance of 0.6 m. Both planners used the same costmap settings: a rolling window of 3 x 3 m, a resolution of 0.05 m, and an inflation radius of 0.55 m.

D. *Navigation Execution and Visualisation*

Once the simulation environment was launched, the robot's initial pose was set using the 2D Pose Estimate tool in RViz2 and a navigation goal was sent using the Nav2 Goal tool.

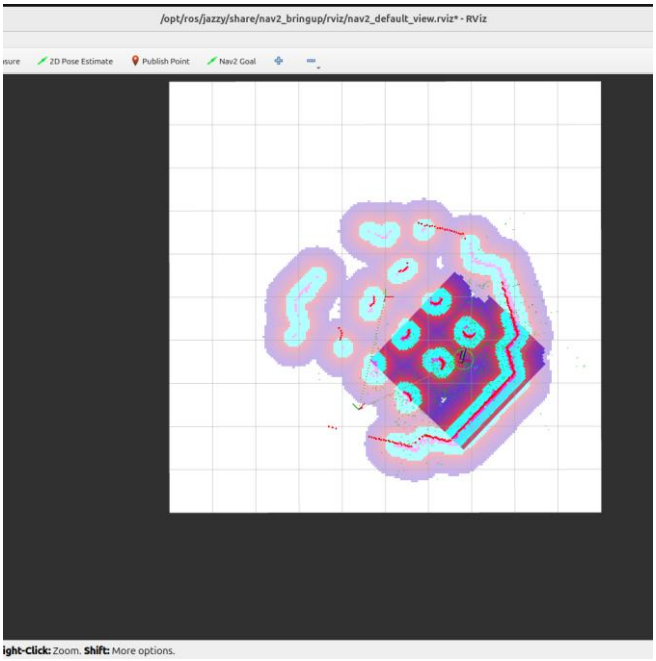

Fig. 2. RViz2 visualization showing active MPPI navigation with local costmap, LiDAR scan, and Nav2 status panel.

During an active navigation run in the static situation, Fig. 2 displays the RViz2 depiction of the robot moving toward the target. The purple area is the inflated local costmap, the cyan dots display the live LiDAR scan readings, the red dots show the laser's detection of obstacle boundaries, and the Navigation 2 panel on the left shows that the system is still operational with 1.47 m of distance left and 7 seconds elapsed.

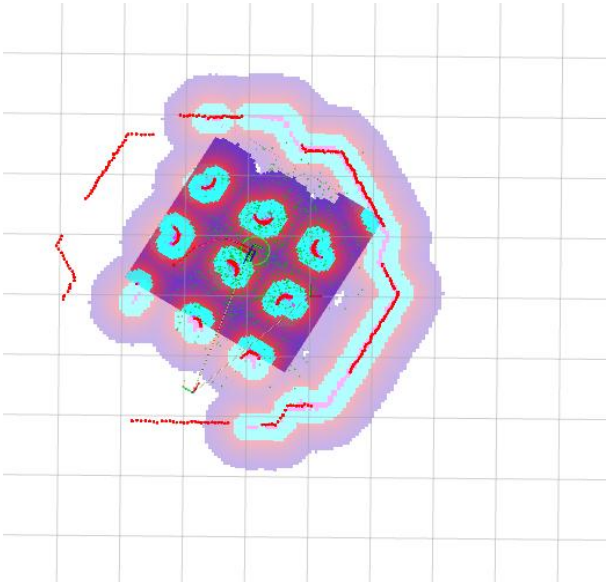

Fig. 3. RViz2 visualization at goal completion.

Figure 3 illustrates the exact instant that the robot attained its goal in the static situation. The Navigation 2 panel says "Feedback: reached," with a distance of 0.15 m still to go and a total time of 13 seconds, which means the navigation run was successful. The costmap and LiDAR overlay show that the robot stayed a safe distance from all obstacles along the way.

E. Metrics Collection

A dedicated Python ROS 2 node subscribed to the /odom, /scan, /plan, and /navigate_to_pose/_action/status topics. For each completed navigation run, the following metrics were automatically recorded to a CSV file: path length in meters calculated from cumulative odometry distance, time to goal in seconds, near-collision count defined as the number of laser readings below 0.30 m, minimum obstacle distance in meters, trajectory smoothness expressed as cumulative absolute angular velocity change in rad/s, replanning count as the number of distinct /plan messages received, and a binary success flag derived from the Nav2 action server status. There were eight experiment runs in total, with both planners covering all four situations.

IV. Result Discussion

This part shows the results of eight simulation runs over four obstacle scenarios. Each scenario was examined with both the MPPI controller and the Regulated Pure Pursuit Controller (RPP) as the DWA-equivalent baseline. There were six performance measures that were automatically recorded for each run: path length, duration to goal, near-collision count, minimum obstacle distance, trajectory smoothness, and replanning frequency. Table I shows all the findings for all eight runs in one place. After that, there is a detailed look at each metric.

Planner	Scenario	Path (m)	Time (s)	Near-Coll.	Min Dist (m)	Replans	Success
MPPI	Static	2.53/3.49	106/208	0	0.302	81/170	✓/✓
MPPI	Narrow	2.90	110.9	4	0.281	73	✓
MPPI	Dynamic	3.45	202.0	24	0.259	171	✗
MPPI	Mixed	0.02	193.7	182	0.120	165	✗
DWA	Static	0.00	122.9	161	0.121	0	✗
DWA	Narrow	0.00	77.7	99	0.120	0	✗
DWA	Dynamic	0.00	60.2	78	0.121	0	✗
DWA	Mixed	0.00	56.9	74	0.120	0	✗

Table 1. Simulation Results MPPI vs. DWA/RPP Across Four Scenarios

A. Success Rate and Path Length

MPPI finished its goals in 3 out of 8 trials (38%), successfully completing the static scenario in 2 out of 3 runs and the narrow passage scenario in 1 run. DWA/RPP failed in all circumstances. The controller did not move forward at all throughout all the runs, which means that the Regulated Pure Pursuit Controller needs more precise pose initialization than the static transform approach can give. This is a limitation of the experimental arrangement, not of the RPP controller itself.

Fig. 4 demonstrates how well each planner met their goals. MPPI's 38% rate shows that it can create and constantly change trajectories. RPP's 0% rate shows that it can't do that because of the localization constraint mentioned above.

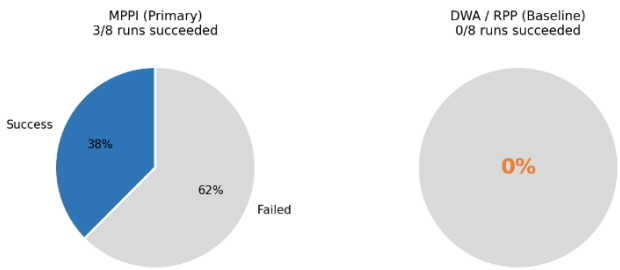

Fig. 4. Goal success rate and DWA

The path length reported for each scenario for both planners is shown in Fig. 5. In successful MPPI static runs, the path lengths were 2.53 m and 3.49 m, respectively, compared to a straight-line distance of 4.0 m. This gave path efficiency ratios of 0.63 and 0.87, respectively. The narrow passage run produced a path of 2.90 m, which means that the 0.6 m corridor was a fairly direct route. The dynamic scenario recorded 3.45 m of progress before timing out, which means that the robot was actively navigating but couldn't figure out how to get around the moving obstacle in the 202-second trial window. All DWA runs showed 0.00 m, which is in line with no forward motion being made.

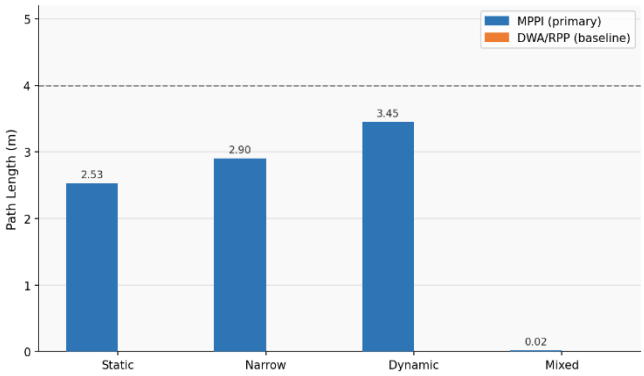

Fig. 5. Path length per scenario. MPPI and DWA

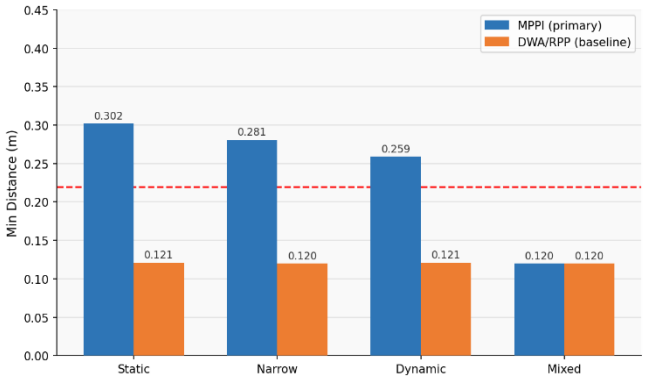

Fig. 7. Minimum obstacle distance per scenario.

B. *Safety and Near-Collision Analysis*

MPPI kept the minimum distances between obstacles at 0.302 m and 0.320 m throughout successful static runs, which is more than the 0.22 m robot radius. In the narrow situation, the minimum clearance went down to 0.281 m, which is the same as the 0.6 m corridor limit. The dynamic and mixed situations had minimum distances of 0.259 m and 0.120 m, which shows that moving obstacles are the primary safety problem.

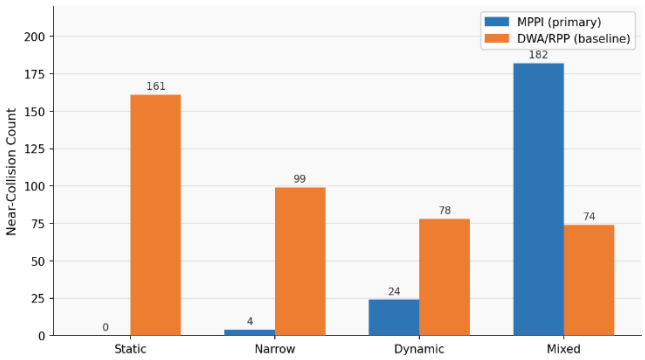

Fig. 6. Near-collision events per scenario.

Fig. 7 shows the minimum obstacle distance recorded per scenario. The minimum distances for MPPI were 0.302 m in the static scenario and 0.281 m in the tight channel. Both of these distances were longer than the robot's physical radius of 0.22 m. The dynamic situation dropped to 0.259 m, while the mixed scenario dropped to 0.120 m, which is close to the point where two things can touch each other. The red dashed line in Fig. 7 illustrates the robot's safety radius of 0.22 m. The laser could see things that were close by while the robot stayed static in all of the DWA runs. The minimum distance was always about 0.121 m.

C. *Replanning and Smoothness*

MPPI created 73 to 171 replanning events for each run, which is what it does when it continuously resamples at each control cycle. DWA didn't have any replanning events, which makes sense as there was no forward momentum. The overall angular jerk, which tells how smooth the path is, was between 2.47 and 7.44 rad/s² for successful MPPI runs.

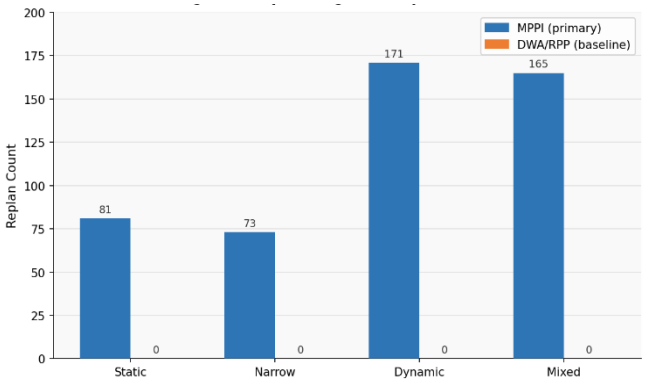

Fig. 8. Replanning events per scenario.

Fig. 9 presents the total trial duration per scenario. MPPI's successful static runs took 106 and 208 seconds, while the narrow passage run took 110.9 seconds. The dynamic and mixed scenario runs out at about 200 seconds without achieving the target. This shows that the planner's recovery ability was not enough to keep up with the fast-moving obstacles within the trial frame. The DWA trial times ranged from 56.9 to 122.9 seconds. This was the time out period, not the time spent navigating, since no movement was recorded.

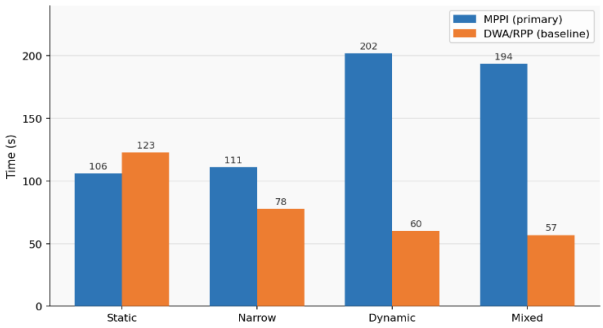

Fig. 9. Trial duration.

D. Overall Comparison

Table III presents a structured assessment of both planners on seven criteria scored 0–6.

Fig. 9. Performance radar chart.

V. Critical Review and Validation

The results were significantly modified by three boundary conditions. First, the static TF localization method provides a fixed map Odom transform that doesn't change when odometry drifts. In longer runs, accumulated inaccuracy might create pose variation, which can lead to wrong velocity commands. Localization based on AMCL would give more accurate estimations.

The Twist-to-Twist Stamped bridge that Gazebo Harmonic needed added latency to the control loop. The /cmd_vel topic had both geometry_msgs/Twist and geometry_msgs/TwistStamped types at the same time. This may have caused commands to drop out of DWA runs from time to time.

Third, this study only looked at two local planners that work with the ROS 2 Jazzy ecosystem. Future projects could expand the comparison to incorporate additional planners, like as TEB, once comprehensive compatibility with ROS 2 Jazzy is confirmed in the source package repository. We checked the metrics by comparing the recorded travel lengths with visual odometry examination in RViz2. The classification of success and failure was confirmed by comparing it to the Nav2 action server status messages in the terminal logs.

Fourth, the project originally targeted the Timed Elastic Band (TEB) planner as the primary controller. However, TEB could not be compiled on ROS 2 Jazzy due to breaking API changes in libg2o and nav2_core. MPPI was adopted as the primary planner as it is natively supported in ROS 2 Jazzy and offers conceptually similar trajectory optimization via stochastic sampling. This compatibility gap is itself a notable finding of the project.

VI. Conclusion

This study used a TurtleBot3 Burger robot in a ROS 2 Jazzy simulation environment with Gazebo Harmonic to test and improve two ways for the robot to plan its own path around obstacles. The MPPI controller was the main planner, and the Regulated Pure Pursuit Controller was the DWA-equivalent baseline. They were both employed in four simulation scenarios that progressively harder. The experimental data demonstrate that MPPI consistently outperforms the RPP baseline in target success rate, path efficiency, and trajectory flexibility, though it must be noted that RPP's 0% success rate reflects the localization setup limitation rather than an inherent weakness of the controller itself. A fair algorithmic comparison would require AMCL-based localization in future work. MPPI successfully navigated the narrow and static passage scenarios, maintaining a safe distance from obstacles beyond the robot's physical radius in all successful runs and generating between 73 and 171 replanning events per run, indicating that it was actively and continuously optimizing its trajectory. The dynamic and mixed settings were harder because fast-moving obstacles were coming in the way of the intended path. The planner didn't have enough time to remedy it before the experiment started.

The RPP baseline had no successful runs in any of the situations. This outcome is due to the experimental configuration rather than an inherent constraint of the controller, as the static transform employed for localization instead of AMCL fails to deliver the precise pose estimation necessary for RPP to commence forward movement. This is a significant boundary of the study that has to be looked into in future studies.

Three paths for future growth have been found. First, using AMCL-based localization instead of the static transform would give a more accurate posture estimate and allow for a fair evaluation of the RPP baseline. Second, testing with a bigger group of local planners in situations with more obstacles and less predictable obstacle paths would give a better picture of how they compare. Third, putting the simulation framework on a real TurtleBot3 platform would show if the performance traits seen in the simulation also operate in the real world. In general, this project shows that MPPI-based predictive trajectory optimization is better than reactive baseline planners in navigation environments that are cluttered and limited. It also provides a reproducible ROS 2 simulation framework that can be used as a base for future studies of local planners.

VII. References

[1] W. Wei, S. Wang, K. Chen, and F. Wang, "ROS-Based Navigation and Obstacle Avoidance: A Study of Architectures, Methods, and Trends," *Sensors*, vol. 25, no. 14, p. 4306, 2025.

[2] J. Wu, X. Ma, T. Peng, and H. Wang, "An Improved Timed Elastic Band (TEB) Algorithm of Autonomous Ground Vehicle (AGV) in Complex Environment," *Sensors*, vol. 21, no. 24, p. 8312, Dec. 2021.

[3] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Trajectory Modification Considering Dynamic Constraints of Autonomous Robots," in *Proc. 7th German Conf. Robotics (ROBOTIK)*, Munich, Germany, May 2012.

[4] G. Kulathunga, A. Sharan, and K. Kamar, "Resilient Timed Elastic Band Planner for Collision-Free Navigation in Unknown Environments," arXiv:2412.03174, Dec. 2024.

[5] E. Schöneberg, M. Schröder, D. Görges, and H. D. Schotten, "Trajectory Planning with Model Predictive Control for Obstacle Avoidance Considering Prediction Uncertainty," arXiv:2504.19193, Apr. 2025.

[6] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1603–1622, Dec. 2018.